Threadmark: source-backed working memory for evolving workflows

SaoLabs AI · Technical research note · 27 October 2026

Status: experimental engineering research, not a peer-reviewed paper. This note documents the released typed-state method, the original frozen experiment, and a separate proposed neural extension. It makes no novelty or equal-accuracy claim. See the references, implementation architecture, and benchmark provenance.

Abstract

Threadmark maintains a bounded register of changing workflow facts and their source identifiers. A question selects cited original records and a recent-record window; a fixed reader answers from those originals. The archive remains available. On 90 synthetic workflows and 360 paired questions, the original implementation achieved 346/360 exact answers versus 349/360 for a full-history reader. Known reader input fell by 79.99%, while known lifecycle input, including memory construction, fell by 38.91%. These results do not establish equal accuracy or guaranteed cost savings. The reusable release refactors this method; it has not rerun the original live experiment. A future latent-memory writer connected to a frozen decoder is a proposal, not implemented functionality.

1. The idea in ordinary language

A project starts with Maya as owner and Friday as its approved deadline. An approved update transfers ownership to Sam. Another record proposes Tuesday without approval. The useful current answer is **Sam / Friday**.

Threadmark keeps a register with those fields and links to the relevant decisions. When asked for the owner and deadline, it retrieves original decisions plus recent records. The reader checks those records and answers with citations. The register's predicted values are not included in the reader's input. An incorrect citation can still retrieve the wrong record, and a recent record can sometimes rescue a mistaken register.

This shifts repeated reading into an ingestion step. It does not replace original storage, make model weights learn online, or guarantee complete recall.

2. Notation and state

Symbol	Meaning
$A_i = (e_1, \ldots, e_{t_i})$	Retained chronological archive through ingestion batch i
C	Public catalog of entities, fields, and their definitions
K	Catalog's set of tracked entity-field keys
s_i	Predicted register after ingestion batch i
ΔA_i	Next whole-event chronological batch
$U_{\Theta_U}, R_{\Theta_R}$	Fixed updater and reader models with their prompts
q, F_q	Public question and requested entity-field keys
$E_i(q)$	Selected original evidence packet
V	Structural validator; may reject a candidate

A register contains one entry per key:

$$s_i = \{(k, v_{i,k}, I_{i,k}) : k \in K\}.$$

Here v is a bounded string or `null`, and I contains at most two original event IDs. The release supports at most 32 tracked facts; memory is capped at 12,000 UTF-8 bytes and each value at 256 characters. A never-established field starts as `null` without evidence. An explicitly cleared field can be `null` with evidence.

The update is:

$$\widetilde{s}_i = U_{\Theta_U}(C, s_{i-1}, \Delta A_i), \qquad s_i = V(\widetilde{s}_i; K, A_i).$$

Both parameter sets stay fixed during normal operation:

$$\Delta\Theta_U = \Delta\Theta_R = 0.$$

The external state and archive change; model parameters do not. The updater is instructed to apply explicit authorized decisions in order, keep proposals distinct from approvals, and change only the fields actually set. These are model instructions, not a proof of correct interpretation.

The validator enforces JSON shape, exact catalog membership, field coverage, bounded values, and citation membership in permitted originals. It cannot prove that a cited event supports a value or concerns the correct entity. Fact correctness therefore requires separate evaluation.

3. Ingestion and original-source selection

Batches preserve complete events. A batch's serialized event envelope must fit the 2,000-token `o200k_base` limit; a single oversized event is rejected rather than split or truncated. This limit covers the event envelope, not the entire API request: the catalog, previous register, and instructions add input tokens. Updater output is limited to 1,024 tokens.

Let W_i be the newest consecutive whole-event suffix whose serialized source envelope fits 1,024 tokens. For the requested fields, gather predicted citations:

$$I_i(q) = \bigcup_{k \in F_q} I_{i,k}.$$

Then select the union in original archive order:

$$E_i(q) = \mathrm{orderedUnique}\left(\{e \in A_i : \mathrm{id}(e) \in I_i(q)\} \cup W_i\right).$$

The serialized union must be at most 20,000 UTF-8 bytes. This byte cap is separate from the recent-window token cap. An overflow fails selection; the implementation does not silently drop cited evidence. The reader receives `source_events`, the catalog, question, and requested keys:

$$\widehat{a}^{\mathrm{TM}}(q) = R_{\Theta_R}(C, q, F_q, E_i(q)).$$

It receives no stored predicted fact values. The full-history baseline is:

$$\widehat{a}^{\mathrm{full}}(q) = R_{\Theta_R}(C, q, F_q, A_i).$$

Reader instructions, model, and 256-token output limit are shared. Citation validation admits only IDs actually supplied to that reader call. Gold state, expected answers, and evaluator annotations are excluded by explicit public-input allowlists; scoring happens after inference.

```
candidate := copy(committed_register)
for batch in chronological_whole_event_batches(new_events):
    candidate := validate(updater(catalog, candidate, batch))
commit(original_archive + new_events, candidate)

for question in questions:
    sources := cited_originals(candidate, requested_fields(question))
    sources := ordered_union(sources, recent_originals(archive))
    require serialized_bytes(sources) <= 20000
    answer := validate(reader(catalog, question, sources))
```

The release commits archive and register together only after every batch in one `ingest` call validates. A later failure leaves the prior committed instance state intact, although earlier provider calls may already have incurred charges. This is an in-process commit rule, not a distributed transaction or durable checkpoint service. One writer per instance is supported.

There are no automatic paid retries or paid full-history fallbacks. An uncertain provider outcome or budget failure halts that provider; a known-usage incomplete answer is

recorded as a failure without retry. Explicit snapshots support restoration, but structural checks do not authenticate an untrusted snapshot. The portable sequential evaluator differs operationally from the original concurrent runner.

4. Lifecycle cost and spending bounds

For one ingested workflow and Q later questions, let $c_U(i)$ be updater cost, $c_F(j)$ full-history reader cost, and $c_E(j)$ evidence-reader cost:

$$C_{\text{full}}(Q) = \sum_{j=1}^{Q} c_F(j), \qquad C_{\text{TM}}(Q) = \sum_{i} c_U(i) + \sum_{j=1}^{Q} c_E(j).$$

If the history stays unchanged, average per-question costs are constant, and $\bar{c}_F > \bar{c}_E$, a simplified break-even condition is:

$$Q \geq \frac{C_U}{\bar{c}_F - \bar{c}_E}.$$

For integer Q, round up. Changing histories, additional ingestion, cache conditions, output lengths, prices, and unequal availability can invalidate that approximation. It is not a forecast derived from reader token reduction alone.

The original study's known spend across all 90 workflows was:

Questions per workflow	Full history	Threadmark, including all ingestion
First question	$0.226120	$0.661341
Four questions	$0.902668	$0.8168903

Total known ingestion cost was $0.611419. The totals exclude charges for three unknown-usage failures and exclude development spend of $0.18242976. They are observed test totals, not provider invoices or guaranteed savings.

Before transport, the budget ledger requires:

$$C_{\text{known}} + H_{\text{pending}} + H_{\text{unknown}} + r_{\text{new}} \leq B.$$

The default reservation uses a conservative UTF-8 byte input bound plus 1,024 framing units and the maximum output allowance:

$$r = 10^{-6}\left[(b_{\text{instructions + input}} + 1024)\max(p_{\text{input}}, p_{\text{cached}}, p_{\text{write}}) + o_{\max} p_{\text{output}}\right].$$

Rates are caller-supplied dollars per million tokens. Valid known usage settles the reservation; unknown usage retains its full hold instead of counting as zero. An observed overrun is recorded and halts the provider. This local guard depends on correct rates and bounds and is not an account-wide provider spending limit.

5. Evaluation and observed results

The frozen October 27 experiment contained 90 synthetic workflows: meetings, support, and projects, each with 10 instances in each of three nominal history bands. Actual rendered histories ranged from 1,846 to 15,215 estimated tokens. Each workflow had two entities, 14 or 16 tracked facts, and four final-state questions. The updater and both readers used `gpt-5.6-luna`. Development informed the settings before fresh test seeds were generated; scenario families and a finite contextual-paragraph pool were shared.

For planned question j with requested keys F_j, exact answer credit requires every returned value to equal its label without normalization:

$$z_j = \mathbf{1}[\text{available}_j] \prod_{k \in F_j} \mathbf{1}[\widehat{v}_{j,k} = v^*_{j,k}], \qquad \text{Accuracy} = \frac{\sum_j z_j}{N_{\text{planned}}}.$$

An unavailable answer scores zero. Evidence credit additionally requires exact equality of the predicted and expected citation sets for every requested field. Valid citation syntax alone earns no semantic credit. Register field accuracy is scored separately.

Result	Full history	Threadmark evidence
Exact values, all planned questions	349/360 (96.94%)	346/360 (96.11%)
Exact values and expected evidence	347/360 (96.39%)	343/360 (95.28%)
Available answers	360/360	354/360
Meeting exact answers	116/120	113/120
Support exact answers	116/120	118/120
Project exact answers	117/120	115/120

There were 338 questions correct in both arms, **11 regressions** (full correct, Threadmark wrong or unavailable), **8 improvements**, and 3 wrong in both. Threadmark's 14 failed answers comprised 8 semantic errors and 6 unavailable answers. Three unknown-usage API failures - one updater and two reader requests - caused those six unavailable answers. They remain in the denominator. No failed request was retried; an operational continuation dispatched previously unattempted requests while retaining original failures.

Register accuracy was **1,336/1,380 fields (96.81%)**, including failed pipelines. Known reader input fell 79.99%; including updater input, reduction was 38.91%. Completion coverage differs, so cost and token comparisons require that qualification. Median recorded reader API time was 2.33 seconds for full history and 2.20 seconds for Threadmark; median ingestion time was 22.89 seconds. Concurrent execution, interruption, and changed cache conditions prevent production latency claims.

The checked-in summary, public evidence archive, and provenance guide document the original frozen run. These are not measurements of a fresh refactored-release reproduction.

6. A proposed latent VSLM and decoder connection

This section describes a **future experiment**, absent from the current release and benchmark. Its foundations are recurrent segment memory [1], learned context compression [2], continuous conditioning of frozen models [3], and generated latent memory [4]. None was reproduced by the typed-register experiment.

A trainable small writer could update a bounded matrix $M_i \in \mathbb{R}^{K_m \times d}$:

$$M_i = U_\theta(M_{i-1}, x_i), \qquad M_0 = 0.$$

A candidate gated realization is:

$$C_i = \tanh(H_i W_C + b_C), \quad G_i = \sigma([M_{i-1} \| H_i] W_G + b_G),$$

$$M_i = (1 - G_i) \odot M_{i-1} + G_i \odot C_i.$$

$H_i \in \mathbb{R}^{K_m \times d}$ is a causal writer's output at learned write-query positions after the previous memory and new event chunk. Concatenation $\|$ joins features, so $W_C \in \mathbb{R}^{d \times d}$, $W_G \in \mathbb{R}^{2d \times d}$, and both biases have width d and broadcast across slots. Bounded values provide numerical stability, not factual guarantees. An illustrative configuration is 32 slots of width 256, projected into a decoder of width D:

$$P_i = \mathrm{RMS}_0(M_i) W_P, \qquad W_P \in \mathbb{R}^{d \times D}.$$

Row-wise normalization means $\mathrm{RMS}_0(m) = m / \sqrt{d^{-1} \sum_{a=1}^{d} m_a^2 + 10^{-6}}$, without an affine gain or bias. A matching vector width alone does not establish decoder alignment; the projection must be trained.

These would be history-dependent **input soft tokens**, not Prefix-Tuning's optimized all-layer prefix activations. A frozen decoder with weights Θ could generate:

$$p_{\theta,\phi;\Theta}(y \mid A, q) = \prod_{j=1}^{|y|} p_\Theta(y_j \mid S, P_\phi(M_i), R_i, E_i, q, y_{<j}).$$

Here R_i, E_i are bounded recent text and retrieved evidence, and S is system framing. Let $\phi = \{W_P\}$ and let N_y count all target response tokens in the training examples. A response loss could train the writer and projection while freezing the decoder:

$$\mathcal{L} = -\frac{1}{N_y} \sum_{t,j} \log p_{\theta,\phi;\Theta}(y^*_{t,j} \mid A_t, q_t, y^*_{t,<j}),$$

$$(\theta, \phi) \leftarrow (\theta, \phi) - \eta \nabla_{\theta,\phi} \mathcal{L}, \qquad \Delta\Theta = 0.$$

Gradients must still flow through the decoder operations to P, M, and θ. A frozen decoder forward pass inside `no_grad()` would prevent that response-based training. Future events

and answer targets must not leak into the historical memory. Memory stays fixed while generating an answer; changing its prefix invalidates dependent decoder KV caches.

This requires access to input embeddings, positions, and masks, which an ordinary text API does not expose. ICAE's encoder reuses a large language model; its small added parameter count does not establish a tiny standalone writer. The proposed VSLM needs independent training and fresh comparisons against retrieval, summaries, and caching.

7. Limits and next evidence needed

Fixed finite memory cannot encode unlimited arbitrary history exactly. The current implementation retains originals and targets explicitly cataloged current fields. It does not establish arbitrary historical reasoning, prompt-injection resistance, production isolation, or superiority over simpler retrieval. No retrieval-only, keyword, embedding-search, or heuristic-selection baseline was run, so the updater's incremental benefit remains unisolated. The earlier trained 0.6B weekday classifier is a separate narrow study.

The next experiment should freeze an accuracy margin and lifecycle budget, compare simpler selectors on fresh independent workflow families, and include real authorized tasks, chronological queries, failure coverage, and cache-controlled latency. The proposed latent writer additionally needs ablations that remove or shuffle memory to test whether the decoder uses it.

References

1. Bulatov, Kuratov, and Burtsev. *Recurrent Memory Transformer* (2022). Primary paper. Recurrent learned memory across segments.

2. Ge et al. *In-context Autoencoder for Context Compression in a Large Language Model* (2023; ICLR 2024). Primary paper. Learned compressed memory and a frozen decoder.

3. Li and Liang. *Prefix-Tuning: Optimizing Continuous Prompts for Generation* (2021). Primary paper. Continuous prefixes with frozen model weights.

4. Zhang, Fu, and Yan. *MemGen: Weaving Generative Latent Memory for Self-Evolving Agents* (2025). Primary paper. A learned memory trigger and weaver.

See the annotated bibliography and BibTeX entries for full attribution and how each source relates to this project.